

CARRERA:

TECNICO SUPERIOR EN DESARROLLO DE SOFTWARE

DESARROLLO CURRICULAR ÁULICO DEL MÓDULO / TALLER / ASIGNATURA /

TALLER DE DISEÑO DE SOFTWAREN I

Curso: Primer Año

Año: 2026

Régimen: ANUAL

TURNO: Tarde / Noche

HORAS: Tarde (4 hs)

MODALIDADES: Presencial



PROF. ING. DANIEL MALDONADO

SAN FERNANDO DEL VALLE DE CATAMARCA,

FIRMA Y ACLARACIÓN DEL PERSONAL QUE LO RECIBE:



2- SINTESIS EXPLICATIVA

La Industria de la Ingeniería del Software y el desarrollo de aplicaciones es la actividad económica que mayor valor agregado genera. Todos los Países Desarrollados del mundo tienen como objetivo fundamental promover esta actividad económica que se basa en algo tan rico como son los recursos humanos capacitados y la actividad netamente intelectual.

Argentina a través de diferentes planes nacionales y gubernamentales tiene la intención de promover la Industria del Software brindando capacitaciones, líneas de créditos flexibles, leyes nacionales y acuerdos internacionales. Sin lugar a dudas La Industria del Software marcará en unos años la diferencia entre países desarrollados y subdesarrollados.

Los Técnicos en Desarrollo de Software tienen un gran campo de acción laboral no tan solo en el presente sino también en el futuro.

Es imprescindible que los futuros egresados del Instituto Superior General San Martín conozcan profundamente como desarrollar software, como realizar programas fácilmente adaptables con código reutilizable, que conozcan un lenguaje de Programación que les permita integrarse al mundo de la Ingeniería del Software, que puedan integrarse a departamentos y organizaciones que producen soluciones informáticas, que conozcan la arquitectura de las aplicaciones, el rol que tienen actualmente los programadores y su amplio campo laboral.

Sin dudas los alumnos que decidan dedicarse Industria del Software tendrán grandes e interesantes ofertas laborales; la Institución y los docentes brindarán todo el contenido, la orientación, la capacitación y la experiencia profesional con el objetivo de formar futuros técnicos con sólidos conocimientos.



3- **a- Objetivos Generales**

- Brindar una introducción a las herramientas metodológicas necesarias para el desarrollo de software mediante el paradigma de la Programación Modular.
- Lograr que el estudiante adquiera aptitud en la resolución de problemas a través del desarrollo de algoritmos.
- Que el estudiante logre autonomía y pueda explorar en forma independiente las posibilidades que ofrecen los distintos lenguajes de Programación.
- Que el estudiante pueda evaluar el resultado de sus soluciones utilizando su capacidad reflexiva y crítica.
- Que el alumno aprenda el oficio de la Programación, que tenga su mente abierta a los cambios, que busque las mejores soluciones a un determinado problema.
- Que logre integrarse a las Organizaciones donde desempeñará su función, interprete los alcances de un profesional de Sistemas dedicado a la programación.
- Que el alumno logre interpretar como se resuelven los problemas de programación, como encarar los proyectos, como dividir grandes problemas en problemas pequeños y así buscar soluciones a un todo.
- El alumno debe reconocer las Estructuras de Datos existentes, los algoritmos que permiten la manipulación de datos con esas estructuras, cuando y como utilizar cada estructura de datos.
- Adquirir los conocimientos necesarios para almacenar datos en las distintas estructuras de datos como una representación conceptual de organizar los datos.
- Comprender las nociones de abstracción de datos y manejo de tipos de datos abstractos.

a- Objetivos Específicos

- Al final el cursado el alumno podrá pensar, analizar y resolver los grandes problemas de programación en problemas más pequeños que pueden ser afrontados de forma aislada. La resolución del problema global no es más que la resolución de los sub-problemas analizados de forma aislada y trabajando de forma colaborativa.
- El alumno podrá comprender el concepto de la programación modular, cuando, como y donde se deben implementar procedimientos y/o funciones. Como invocar funciones desde el programa principal y hasta inclusive la invocación desde otras funciones, los valores que devuelven las funciones, los parámetros enviados por valor y por referencia.
- El alumno dominará el concepto más importante aplicado en varios paradigmas de programación como lo es la resolución y análisis de sub-problemas.
- Al final del cursado el alumno tendrá los conocimientos necesarios y suficientes para desarrollar programas en lenguaje Javascript, utilizando como interfaz HTML y CSS



4- Contenidos del espacio curricular módulo/taller/asignatura

- Contenidos Conceptuales

NÚCLEO TEMÁTICO I — Introducción: La Web y HTML básico

La Web

- Qué es Internet y qué es la Web — diferencias fundamentales.
- Cliente y Servidor — roles y responsabilidades.
- Request y Response — el ciclo de comunicación web.
- HTTP y HTTPS — protocolo de transferencia y seguridad.
- Dirección IP — identificación de dispositivos en la red.
- Servidor DNS — traducción de dominios a direcciones IP.
- El navegador web — función e importancia.
- La URL y sus partes — protocolo, dominio, ruta.
- Dominio y Hosting — qué son y para qué sirven.
- Las 3 tecnologías fundacionales: HTML, CSS y JavaScript.
- Programador Web vs Diseñador Web — diferencias de rol.
- Frontend y Backend — arquitectura de una aplicación web.
- Herramientas de trabajo: Visual Studio Code y extensión Live Server.

HTML básico

- Estructura de un documento HTML — anatomía completa.
- Etiquetas de apertura y cierre — sintaxis fundamental.
- Etiquetas estructurales: `<!DOCTYPE>`, `<html>`, `<head>`, `<body>`.
- Etiquetas de texto: `<h1>` a `<h6>`, `<p>`.
- Etiquetas de formulario básicas: `<input>`, `<button>`, `<label>`, `<form>`, `<select>`.

NÚCLEO TEMÁTICO II — Fundamentos de JavaScript

Introducción

- Qué es JavaScript y por qué es un lenguaje de programación profesional.
- Vinculación del archivo JS con el documento HTML.

Variables y tipos de datos

- Declaración de variables: `let` y `const` — diferencias y buenas prácticas.
- Tipos de datos: `Number`, `String`, `Boolean`, `null`, `undefined`.
- Scope básico de bloque `{}` — variables que nacen y viven dentro de su bloque.
- Conversión de tipos: `Number()`, `String()`, coerción implícita.
- Operador `typeof` — inspección del tipo de una variable.

Operadores

- Operadores aritméticos: `+`, `-`, `*`, `/`, `%`, `**`.
- Operadores de asignación: `=`, `+=`, `-=`, `*=`, `/=`.



- Concatenación de strings con +.

Entrada y salida

- `console.log()` — mostrar resultados en consola.
- `prompt()` — solicitar datos al usuario.
- `alert()` — mostrar mensajes al usuario.
- Template literals — backticks e interpolación con `$`{}``.

Cierre del núcleo — Operadores de comparación y lógicos

Base fundamental para las estructuras de control del Núcleo III. Una expresión de comparación o lógica siempre evalúa a `true` o `false`.

- Operadores de comparación: `==`, `===`, `!=`, `!==`, `>`, `<`, `>=`, `<=`.
- Operadores lógicos: `&&` (AND), `||` (OR), `!` (NOT).
- Diferencia entre `==` (igualdad de valor) y `===` (igualdad estricta de valor y tipo).

NÚCLEO TEMÁTICO III — Estructuras de Control

Estructuras condicionales

- `if`, `else if`, `else` — variables continuas con infinitos valores posibles en un rango.
- Operador ternario — exactamente dos posibilidades: `condicion ? valorTrue : valorFalse`.
- `switch` — variables discretas con valores finitos y enumerables.
- Variables continuas vs variables discretas — cuándo usar `if/else` y cuándo usar `switch`.

Estructuras repetitivas

- `for` — cuando se conoce la cantidad de iteraciones.
- `while` — cuando la condición de corte es dinámica.
- Contador y acumulador como patrones fundamentales dentro de los bucles.
- Operador módulo `%` aplicado a lógica — par/impar, divisibilidad.

Particularidades de los operadores lógicos

- Short circuit `&&` y `||` — evaluación de cortocircuito.
- Uso práctico: asignación de valores por defecto y validaciones simples.

NÚCLEO TEMÁTICO IV — Funciones

Programación modular

- Definición de procedimientos y funciones — por qué dividir el código.
- Concepto de caja negra (funciones puras) — la función como unidad aislada que recibe parámetros, procesa y retorna un resultado sin depender ni modificar nada del exterior.

Definición e invocación

- Declaración de funciones — `function nombreFuncion() {}`.
- Expresión de funciones — `const nombreFuncion = function() {}`.



- Arrow functions — `const nombreFuncion = () => {}`.
- Invocación de funciones.
- Parámetros y argumentos — diferencias.
- Retorno de valores con `return`.
- Valores por defecto en parámetros.

Alcance

- Variables locales y globales.
- Scope en funciones — visibilidad y ciclo de vida de las variables.

Funciones avanzadas

- Rest operator `...` en parámetros de funciones — agrupar argumentos.
- Funciones de orden superior — funciones que reciben o devuelven otras funciones.
- Funciones Callback — función pasada como argumento a otra función.

NÚCLEO TEMÁTICO V — Manejo básico del DOM

Captura y manipulación de elementos

- `querySelector()` — selección de elementos HTML desde JavaScript.
- Lectura de valores de inputs — propiedad `.value`.
- Propiedad `textContent` — mostrar texto plano en un elemento HTML.
- Propiedad `innerHTML` — mostrar contenido HTML dentro de un elemento.
- Diferencias entre `textContent` e `innerHTML` — cuándo usar cada una.
- `classList` — agregar y quitar clases CSS desde JavaScript.

Eventos

- Evento `onclick` — asignar acciones a botones y elementos.
- Delegación de eventos — escuchar eventos en un elemento padre.

Arquitectura y validación

- Integración con funciones — separación entre lógica y presentación.
- Validación básica de formularios desde JavaScript.
- Manejo de errores: `try / catch` — captura y gestión de errores en tiempo de ejecución.
- Módulos: `export` de funciones e `import` desde el controlador — organización profesional del código.

NÚCLEO TEMÁTICO VI — Arrays y Objetos

Arrays — fundamentos

- Creación de arrays — array literal.
- Acceso por índice — base 0.
- Recorrido con `for`.
- Propiedad `length`.



Inserción y eliminación

- push() — insertar al final.
- pop() — eliminar del final.
- unshift() — insertar al principio.
- shift() — eliminar del principio.
- splice() — insertar, eliminar o reemplazar en cualquier posición.

Objetos literales

- Definición de objetos literales — propiedades y valores.
- Acceso a propiedades con punto (.) y corchetes ([]).
- Arrays de objetos literales — estructura de datos fundamental.

Métodos de orden superior

- forEach() — recorrer y ejecutar una acción por cada elemento.
- find() — encontrar el primer elemento que cumple una condición.
- filter() — obtener un nuevo array con los elementos que cumplen una condición.
- map() — transformar cada elemento y obtener un nuevo array.
- some() — verificar si al menos un elemento cumple una condición.
- every() — verificar si todos los elementos cumplen una condición.
- reduce() — acumular valores de un array en un único resultado.

Desestructuración y operadores avanzados

- Desestructuración de arrays — extraer valores en variables.
- Desestructuración de objetos — extraer propiedades en variables.
- Spread operator ... — copiar y expandir arrays y objetos.
- Rest operator ... en desestructuración — agrupar elementos restantes.

- **Contenidos Procedimentales**

- Análisis e Interpretación de los Problemas típicos que se presentan en el ámbito del Diseño de Software.
- Representación de los Problemas y Esquematización de los mismos a través de diagramas de flujo.
- Utilización y práctica de una herramienta gráfica que permite gráficamente poner en práctica las primeras ideas de programación.
- Transformar los Diagramas de Flujo en Pseudocódigo.
- Realizar pruebas de escritorio.
- Escribir los Programas, Compilar y Ejecutar.
- Resolver los Conflictos más comunes que se presentan en el Oficio del Programador.



- Realizar Testing Completo de los programas realizados.
- Corregir los Programas y conseguir la puesta a punto.
- Publicar los Programas terminados.

- **Contenidos Actitudinales (generales para todas las unidades)**
 - Actitud PROACTIVA
 - Constancia y Perseverancia para poder encontrar la solución a los problemas
 - Mente abierta para entender y comprender que un mismo problema puede ser resuelto de diferentes formas.
 - Actitud de Permanente superación; esto significa que el Alumno debe saber que algo que ya resolvió anteriormente puede ser mejorado, haciendo sus soluciones más eficaces y eficientes.
 - Deseo de Superación; Los Lenguajes de Programación son muy ricos y exigen que el Alumno y/o Futuro Técnico esté en permanente capacitación; tanto asistiendo a cursos, capacitaciones presenciales, capacitaciones online, interconsultas en foros de discusión.



- Gran Capacidad de Adaptación; La Industria del Software se caracteriza por la Constante y permanente actualización de ideas, paradigmas, pensamientos, cambios tecnológicos que obligan a que un técnico siga esos nuevos horizontes en busca de poder brindar soluciones actualizadas a sus clientes.

5- Orientaciones Metodológicas

- Clases prácticas
- Clases de Laboratorio
- Evaluación Sumativa
- Trabajos individuales y en grupo

6- Tiempo

Explicitar el tiempo aproximado para el desarrollo de cada unidad

Estos son los tiempos aproximados y estimados por Unidad para Cada Turno.

TALLER Diseño Software - T Tarde		
Cantidad de Clases	Cantidad de Horas Cátedra	Unidad
3	6	Unidad I
15	60	Unidad II
15	60	Unidad III
15	60	Unidad IV
15	60	Unidad V
72	246	



7 – Guías de Trabajos Prácticos

Al ser una materia en formato TALLER, los alumnos deben conseguir el conocimiento a través de la práctica guiada y asistida por parte del docente y de todo el material virtual que el alumno tiene a disposición.

7.1) Material Disponible En línea

- Correo Electrónico Institucional
- Plataforma Educativa Classroom donde tiene contenido actualizado de la Materia
- Para Todas las Modalidades Presencial/Semipresencial/Virtual existe una guía de Trabajos Prácticos extensa que permite al alumno ejercitar y adquirir los conocimientos teóricos y prácticos establecidos en el presente diseño curricular. Sobre cada punto existe un video tutorial online disponible para que el alumno utilice como práctica guiada, corrija sus errores y pueda observar la manera correcta de realizar los ejercicios.
- Exclusivamente para la Modalidad Semipresencial/Virtual el profesor cuenta con un grupo de comunicación What'sapp para una comunicación directa y fluida entre el grupo de alumnos y el profesor para asistirlo en todo momento.

7.2) Guías de Trabajos Prácticos (Modalidades Presencial/SemiPresencial/Virtual)

- Para el Desarrollo completo de la materia, la misma cuenta con 4 (Cuatro Guías de Ejercicios prácticos completamente asistidas y guiadas desde la plataforma Classroom a través de videos Online en la Plataforma Youtube accesibles por internet.



7.3) Ventajas del Material Audio Visual En Línea

Aprendizaje significativo: Tu objetivo principal como docente es que los estudiantes tengan un aprendizaje significativo, es decir, que asimilen la información y la puedan poner en práctica. La educación virtual da acceso a los estudiantes a repasar el contenido cuantas veces lo consideren necesario y desarrollar las actividades a su propio ritmo. La ventaja frente a la educación tradicional es que el estudiante tendrá la posibilidad de repasar tus sesiones sin que esto interrumpa el proceso de los demás estudiantes, lo que a largo plazo será beneficioso tanto para él como para sus compañeros.

Accesibilidad: Permite a Estudiantes acceder al contenido desde cualquier lugar de la provincia disponiendo de una conexión a internet.

Comunicación asincrónica: En la educación virtual, la comunicación entre el estudiante puede ser asincrónica, es decir que la interacción a través de medios digitales se desarrolla en horarios diferidos, evitando que se provoquen retrasos en el proceso de aprendizaje.

Trabajo Colaborativo: La educación virtual facilita el trabajo colaborativo, el acceso a chats, debates y prácticas en las plataformas, enriquecen los conocimientos.

Evaluación Permanente: Un sistema de evaluación continua, en el que el estudiante es consciente y responsable de su proceso de aprendizaje, a la vez que tiene mecanismos alternativos de evaluación diferentes al examen tradicional.



- 8- **Evaluación:** Ver anexo de sistema de evaluación y promoción de asignaturas proporcionadas y estandarizadas por el INSTITUTO SUPERIOR GENERAL SAN MARTIN.

CLASIFICACIÓN	
Según el momento	Según el alcance
Evaluación Inicial	Diagnostica: El profesor durante la etapa de diagnóstico que estará comprendida entre las dos primeras semanas del inicio del ciclo lectivo realizará juegos didácticos que permitirán determinar el grado de conocimientos previos que un alumno posee y son requisitos fundamentales y previos al cursado propiamente dicho de la materia Taller de Diseño de Software. Estos juegos que se desarrollan con una herramienta de diseño de software para niños permite observar si el alumno interpreta los contenidos básicos de cómo está realizado un programa, cual es el diseño, el entorno IDE y la ejecución del mismo.
Evaluación de Proceso	Durante el transcurso del año lectivo, se evaluará al alumno mediante trabajos prácticos, talleres de dos modalidades. <u>Trabajos Prácticos en Clases o Taller:</u> donde el profesor proporcionará una consigna y el alumno en forma individual deberá desarrollar las tareas proporcionadas por el Profesor. Tiempo máximo para desarrollar es de un módulo (máximo dos horas cátedra). <u>Prácticos a desarrollar en Casa:</u> donde el profesor proporcionará una consigna y el alumno en forma individual o grupal deberá desarrollar las tareas proporcionadas por el profesor. Tiempo máximo para desarrollar la actividad es de una semana. <u>Examen Parcial por Unidad:</u> La Materia Consta de 4 (cuatro unidades), cada una de ellas tendrá su examen parcial evaluando todos los contenidos observados en la UNIDAD.
Evaluación Final	<u>Trabajo Práctico Final de Programación en HTML/CSS y Javascript:</u> que será en Equipo (no mayor a tres alumnos) con la intención que se ponga en práctica todo lo estudiado, practicado y analizado durante el año. Este trabajo práctico final es Obligatorio y requisito fundamental para regularizar la materia.



Criterios de evaluación:

Los puntos que a continuación describo son los más importantes que evalúo durante el cuatrimestre y por el cual determino el nivel de conocimiento que el educando adquirió sobre la materia.

- Asistencia del alumno y continuidad en las clases: Es de suma importancia la asistencia y continuidad del alumno a las clases dictadas por el profesor. Dado que los contenidos vertidos en clases son un cúmulo de conocimientos específicos de la materia como así también experiencias profesionales del docente/profesional vinculados directamente. Mucho del material entregado en clases como las experiencias y analogías utilizadas por el profesor no son material que puedan encontrarse fácilmente en un libro, ni en internet, ni en videos. La formación de un futuro profesional técnico informático requiere de docentes que puedan entregar a sus educandos un no tan solo el contenido específico de las Unidades sino también mostrar en el campo real la utilización de los conceptos, las ventajas y/o desventajas de utilizar un método u otro, etc.
- Participación en Clases: Iniciar a los alumnos en el Terreno de la Programación es un proceso de transformación de los educandos. Es decir aquí los alumnos deberán diseñar lógicamente procesos, pensar en soluciones alternativas a las presentadas por el profesor en clases, deberán cuestionar y cuestionarse entre ellos las soluciones que se plantean, deberán discutir e ir formando criterio, pensando y acomodando sus ideas, ordenando mentalmente los pasos a seguir. Es todo un proceso de Transformación, de cambio que implica también resistencia a incorporar esos conceptos. Es importante que el alumno en clases participe, discuta, refleje todas sus dudas e inquietudes individuales y colectivas.
- Presentación de los Trabajos Prácticos: Dentro de este punto el profesor evaluará la presentación en término cumpliendo con las fechas establecidas en el cronograma como así también la prolijidad, eficacia y eficiencia de los programas presentados.
- Interés del Alumno por la Materia: Si bien es un punto difícil de mensurar; el profesor observa el interés del alumno por aprender los conceptos vertidos en clases como así también la preocupación, la motivación que tiene por mejorar, superarse ampliando los conocimientos adquiridos en clases. Dado que el contenido de la Materia es un Pilar básico que el alumno tendrá en su futura carrera profesional es requisito fundamental que el Profesor motive al alumno y este refleje interés por la misma.
- Aprobar los parciales prácticos de la Materia



BIBLIOGRAFÍA BÁSICA: Realizar detalle de la bibliografía obligatoria con apellido y nombre de autor, obra y año de edición.

- Eloquent JavaScript - A Moderne Introduction to Programming
Third Edition
Marijn Haverbeke

BIBLIOGRAFÍA AMPLIATORIA: Realizar detalle de la bibliografía obligatoria con apellido y nombre de autor, obra y año de edición.

- JavaScript for Absolute Beginners
Terry MacNavage

BIBLIOGRAFIA PARA EL DOCENTE: Describir cual es el material bibliográfico que utiliza para desarrollar su asignatura.

- JavaScript-The-Definitive-Guide-6th-Edition
The Definitive Guide
David Flanagan